Argus

Technical Documentation

Grounded Document Question-Answering System

VERSION	0.1.0
REPOSITORY	github.com/Anas-Mirza-22/Argus-Grounded-Document-QA
SCOPE	Architecture · Retrieval · Verification · Security · API · Deployment

Contents

1. Product definition 4

2. Scope 4

3. Architecture 5

3.1 Backend responsibilities . . . 5

3.2 Frontend responsibilities . . . 5

4. Technology stack 6

5. Request lifecycle 6

5.1 Early gates . . . 7

5.2 UNDERSTAND . . . 7

5.3 DECIDE and PLAN . . . 8

5.4 SEARCH and OBSERVE . . . 8

5.5 EVALUATE and REFINE . . . 8

5.6 GENERATE . . . 8

5.7 FINAL . . . 8

6. Retrieval subsystem 8

6.1 Targeted mode . . . 8

6.2 Broad mode . . . 8

6.3 Adaptive top-K . . . 8

6.4 Filtering at query time . . . 9

6.5 Similarity threshold . . . 9

7. Citation protocol 9

7.1 Stable source IDs . . . 9

7.2 Parsing and validation . . . 9

7.3 Frontend rendering . . . 9

7.4 Limitations . . . 9

8. Verification and abstention 10

8.1 Verification signals . . . 10

8.2 Combined verdict . . . 10

8.3 Abstention conditions . . . 10

9. Prompt-injection defenses 10

9.1 Threat model . . . 10

9.2 Documented controls . . . 10

10. Authentication and session management 11

10.1 Supported flows . . . 11

10.2 Exchange-token flow . . . 11
10.3 Session cookie . . . 11
10.4 Password reset . . . 11
10.5 Security considerations . . . 11
11. Data model and persistence 12
12. API reference 12
13. Frontend structure 14
13.1 Routes . . . 14
13.2 Main components . . . 14
13.3 Streaming behavior . . . 14
13.4 Mobile behavior . . . 14
14. Voice and image workflows 15
14.1 Voice input . . . 15
14.2 Voice output . . . 15
14.3 Image-assisted Q&A . . . 15
15. Admin dashboard and metrics 15
16. Deployment 15
16.1 Documented hosting layout . . . 15
16.2 Environment configuration . . . 16
16.3 Health checks . . . 16
16.4 Production checklist . . . 16
17. Local development 16
18. Engineering challenges and lessons 17
19. Known limitations and open risks 18
20. Roadmap 18
21. Glossary 19
22. Contribution guidelines 19
23. License 20

Purpose: Technical reference for the architecture, pipeline, retrieval, evidence validation, security model, API surface, deployment, and development workflow.

> *This document is based on the project notes supplied for Argus. Feature availability, endpoint schemas, environment-variable names, and issue status should be confirmed against the current source code before a production release.*

1. Product definition

Argus is a document-grounded question-answering application. Users upload PDF, TXT, or Markdown files and ask questions about their contents. The system retrieves relevant chunks, generates an answer with inline source identifiers, validates citation references, and evaluates whether the answer is supported by the retrieved evidence.

The design prioritizes:

- Evidence-grounded answers.
- Traceable citations.
- Explicit abstention when support is insufficient.
- A visible pipeline trace.
- User-scoped document and conversation access.
- Defensive handling of untrusted document content.

Argus is not intended to be a general-purpose assistant. Its orchestration is implemented as explicit Python control flow rather than a LangChain agent, tool-calling loop, or LangGraph graph.

2. Scope

In scope

- Document upload and document-scoped retrieval.
- Targeted questions, document summaries, and multi-document comparisons.
- Citation parsing and source validation.
- Grounding checks and verifier-assisted abstention.
- Conversation sessions and exports.
- Authentication and administrative metrics.
- Voice input and image-assisted question extraction.

Out of scope for the documented v1

- General-purpose responses unsupported by uploaded documents.
- Multi-modal answer generation.
- Horizontal scaling across multiple backend replicas.
- Real-time collaborative sessions or shared editing.

3. Architecture

```
Browser
  └── React SPA (Vite)
      ├── Authentication context
      ├── Document and session workspace
      ├── Chat and answer cards
      ├── Agent trace and citation rendering
      ├── Voice controls
      └── Admin dashboard
            │ HTTPS / JSON / SSE
            ▼
FastAPI
  ├── Authentication and request validation
  ├── RAG state pipeline
  ├── Retrieval and citation protocol
  ├── Verification and abstention
  ├── Voice / image processing
  └── Admin and query metrics
       ├── SQLite: users, sessions, messages, documents, metrics
       └── ChromaDB: vector embeddings and chunk metadata
```

3.1 Backend responsibilities

Module	Responsibility
`main.py`	Routes, dependencies, middleware, and API setup
`auth.py`	Password authentication, JWT/session helpers, exchange flow
`google_auth.py`	Google OAuth/OIDC
`password_reset.py`	Password-reset flow
`rate_limit.py`	Per-user sliding-window rate limiting
`validation.py`	Upload validation
`rag_engine.py`	Coordinates the question-answering stages
`retrieval.py`	Vector search, adaptive top-K, and document filtering
`citations.py`	Citation ID parsing, mapping, and source validation
`verification.py`	Grounding signals and final verdict
`routing.py`	Task-to-model mapping
`agent_state.py`	Per-request agent settings
`storage.py`	SQLite persistence
`memory.py`	History and session-summary formatting
`vision.py`	Image-based question extraction
`voice.py`	Audio transcription integration

3.2 Frontend responsibilities

The React application owns route rendering, authentication state, document/session views, settings, chat interactions, SSE consumption, answer display, citation highlighting, feedback, and administrative charts. Server-owned information such as users, documents, sessions, and feedback is persisted through the API; some interface settings are stored locally.

4. Technology stack

Area	Technology
Runtime	Python 3.11
API framework	FastAPI
LLM integration	`langchain-groq` and Groq API
Main generation model	`openai/gpt-oss-120b`
Structured tasks / verifier	`openai/gpt-oss-20b`
Embeddings	`sentence-transformers/all-MiniLM-L6-v2` (384 dimensions)
Vector database	ChromaDB, cosine similarity
Relational database	SQLite
PDF parsing	`pypdf`
PDF generation	ReportLab
Email	Resend
Password hashing	bcrypt
JWT	PyJWT
OAuth	Google OAuth/OIDC
Frontend	React 18, TypeScript 5.6, Vite 5
Routing	`react-router-dom`
Markdown	`react-markdown`, `remark-gfm`, `rehype-raw`
Charts	Recharts
Hosting	Railway, separate frontend and backend services

5. Request lifecycle

The pipeline uses named states and conditional branches. A typical document question follows this flow:



The user-facing trace exposes stage activity. It should be understood as an execution trace, not as a guarantee that every internal model judgment is correct.

5.1 Early gates

The implementation documents early checks for:

- Greetings.
- Capability questions.
- Gratitude.
- Past-session questions.
- Document-overview requests.

These can return a short-circuit response or invoke a specialized overview path without running the full retrieval pipeline.

5.2 UNDERSTAND

A smaller model returns structured data describing the question, including:

- An interpretation or reasoning summary.
- An adversarial flag.
- A standalone rewritten question.
- Whether retrieval is needed.
- The response format.
- A search query.
- A decision summary.
- Whether broad retrieval is appropriate.

If the request is classified as adversarial, the pipeline can abstain before retrieval.

5.3 DECIDE and PLAN

The system selects a response format and retrieval mode, then builds the query used by the retriever. The notes state that selecting two or more documents can promote a targeted query to broad mode.

5.4 SEARCH and OBSERVE

The retriever queries ChromaDB with the active document scope and chosen top-K budget. The OBSERVE stage records the number of chunks and leading sources for the trace.

5.5 EVALUATE and REFINE

The documented implementation treats retrieval as sufficient when at least four chunks are returned; otherwise, it asks the smaller model whether the evidence is sufficient. If the context is insufficient and retry attempts remain, the query is refined and retrieval is repeated.

This is a documented implementation choice, not a universal rule. A high chunk count does not necessarily mean the evidence is relevant or sufficient.

5.6 GENERATE

The main model streams an answer while being instructed to cite retrieved chunks using stable IDs. Retrieved text is wrapped as untrusted context. The answer then passes through a lightweight support check and the fuller verification process.

5.7 FINAL

The result is persisted with the answer, citations, sources, trace, and evaluation metadata. The stream emits a final event and then completes.

6. Retrieval subsystem

6.1 Targeted mode

Targeted retrieval performs a flat query over the selected document scope. It is intended for specific questions where the strongest matching passages matter most.

6.2 Broad mode

Broad retrieval queries selected documents individually to improve coverage. It deduplicates results by chunk identifier, retains the best-scoring duplicate, sorts results, and can backfill from a flat query when fewer than the desired number of chunks are available.

Broad mode is intended for summaries, comparisons, and questions where coverage across documents is important.

6.3 Adaptive top-K

The documented policy calculates a retrieval budget from the base top-K, number of selected documents, and whether broad mode is active. The total is capped at 24 chunks.

```
def effective_k(n_docs, broad, base_top_k):
    base = max(1, base_top_k)
    k = min(base * max(1, n_docs), 24)
    if broad:
        k = min(k + base, 24)
    return max(1, k)
```

This snippet illustrates the policy described in the project notes; use the current implementation as the source of truth.

6.4 Filtering at query time

Document IDs are passed into the vector-store query as metadata filters. Filtering before retrieval is preferable to retrieving from the entire index and discarding unselected results afterward, because post-filtering can waste the top-K budget.

6.5 Similarity threshold

If a positive similarity threshold is configured, results below the threshold are removed. The documented behavior does not silently fall back to unfiltered results if all matches are removed. The default threshold is described as zero because similarity distributions depend on the embedding model and chunking strategy.

7. Citation protocol

7.1 Stable source IDs

Each retrieved chunk is assigned a stable ID for the current answer, for example:

```
[S1] report.pdf — page 3
Relevant source text...

[S2] policy.md
Relevant source text...
```

IDs are local to the current retrieval context; they should not be treated as permanent document identifiers.

7.2 Parsing and validation

The documented process is:

1. Instruct the model to cite only supplied `[Sn]` IDs.
2. Parse citation markers from the generated answer.
3. Split known citation IDs from unknown IDs.
4. Map known IDs back to retrieved chunks.
5. Build citation objects only for referenced chunks.
6. Validate the citation evidence against source text, with a fuzzy fallback.
7. Let verification reject unknown IDs or insufficient evidence.

7.3 Frontend rendering

The frontend highlights citation markers as separate visual elements. Citation parsing and citation rendering should have a single clear owner to avoid duplicate transformations, marker concatenation, or broken links between answer text and source references.

7.4 Limitations

A valid source ID only proves that the source was retrieved, not that it supports the adjacent claim. Citation validation and answer verification are separate responsibilities and should remain separate checks.

8. Verification and abstention

8.1 Verification signals

The supplied documentation describes three primary signals:

1. **Content-token grounding:** compares content words and named entities in the answer with the retrieved context after removing stopwords, punctuation fragments, and short tokens.
2. **Citation coverage:** measures whether retrieved evidence is represented in the answer's citations.
3. **LLM-as-verifier:** asks a separate model whether the answer's claims are supported by the retrieved chunks.

Unknown citation IDs and a support-check failure can also veto an answer.

8.2 Combined verdict

The verifier combines grounding, citation, confidence, unknown-citation, and LLM-verifier outcomes. The implementation is designed to fail closed: a verifier error or malformed response should not be treated as a successful verification.

The documented confidence formula is:

```
confidence = 0.5 × max_retrieval_score + 0.5 × token_coverage
```

The supplied notes specify a default abstention threshold of `0.15`, tuned for the stated embedding model's score distribution. Thresholds should be validated on a representative evaluation set before they are changed.

8.3 Abstention conditions

Abstention can occur when:

- The request is flagged as adversarial.
- No usable chunks are retrieved.
- The support check fails.
- Citation IDs are unknown.
- The combined verification verdict fails after retries.
- Verification fails due to an error.

Canonical abstention text documented by the project:

> *I don't have enough grounded evidence in the provided documents to answer confidently. Please try rephrasing or upload more relevant material.*

Abstention is a safety behavior, but it can also produce false negatives when relevant evidence is phrased differently from the answer. Evaluation should measure both unsupported answers and unnecessary abstentions.

9. Prompt-injection defenses

9.1 Threat model

Uploaded files may contain text that attempts to override instructions, impersonate system messages, or manipulate the model into answering from general knowledge.

9.2 Documented controls

- **Structural wrapping:** retrieved chunks are wrapped in a context block and identified as untrusted document data.

- **Pattern detection:** known injection-like phrases and control tokens can cause chunks to be tagged as untrusted.
- **User-message classification:** the UNDERSTAND stage can flag adversarial instructions and trigger abstention before retrieval.
- **Trace visibility:** flagged inputs and chunks can be recorded for operator review.

Pattern detection is necessarily incomplete: novel or indirect attacks may not match known regular expressions. Context wrapping and classification reduce risk but do not eliminate prompt injection. Multimodal input paths should be tested independently.

10. Authentication and session management

10.1 Supported flows

The documented authentication surface includes:

- Email/password registration and login.
- Google OAuth/OIDC.
- One-time exchange tokens used to establish a session cookie.
- Logout.
- Password-reset requests and reset-token consumption.

10.2 Exchange-token flow

In the split-origin deployment, login or the OAuth callback returns a short-lived exchange token rather than directly setting the final session cookie during a cross-site navigation. The frontend redeems the token through a fetch request, and the backend sets the session cookie in that response.

The notes describe exchange tokens as opaque, single-use, and valid for 120 seconds, held in memory for the single-replica deployment.

10.3 Session cookie

The documented production cookie uses:

- `HttpOnly`
- `Secure`
- `SameSite=None` for production cross-origin requests, and `Lax` in development
- A seven-day maximum age

The JWT includes a user subject and issue/expiry timestamps. The application requires a JWT signing secret; startup should fail if it is absent.

10.4 Password reset

The forgot-password endpoint is designed not to disclose whether an email address exists. Reset tokens are short-lived. Google-only accounts should use Google sign-in rather than a password reset.

10.5 Security considerations

The source notes identify remaining concerns:

- Origin/CSRF checks on the exchange endpoint.
- Binding exchange tokens to the initiating client.
- Incognito behavior in some cross-origin browser configurations.
- Strong user scoping for every document/session read and write.

- Additional injection testing for image and voice workflows.

These items should be addressed or explicitly risk-accepted before handling sensitive data.

11. Data model and persistence

SQLite stores application and analytics records. The documented schema includes:

Table	Purpose
`users`	Account identity, password hash, admin flag, Google subject
`sessions`	Conversation sessions and ownership
`messages`	User/assistant messages and metadata
`documents`	Uploaded document records and processing status
`session_summaries`	Conversation summaries used for history
`auth_events`	Authentication event records
`query_metrics`	Grounding, precision, verification, latency, and abstention metrics
`password_resets`	Reset-token records and expiry
`feedback`	User feedback on assistant messages

ChromaDB stores vector embeddings and chunk metadata. Persistent storage is required if these indexes and records must survive redeployments.

Data isolation principles

- Scope every read and write by the authenticated `user_id`.
- Apply selected-document filters at retrieval time.
- Exclude the current session from past-session summary retrieval when collecting cross-session context.
- Avoid global in-memory document registries as an authorization source.
- Test deletion and ownership checks for both SQLite records and vector-store entries.

12. API reference

The API prefix documented by the project is `/api/v1`. Confirm exact schemas in the live OpenAPI documentation at `/docs`.

Authentication

Method	Path	Purpose
POST	`/auth/register`	Register an account
POST	`/auth/login`	Log in
POST	`/auth/exchange`	Redeem an exchange token
POST	`/auth/logout`	Clear the session cookie
GET	`/auth/me`	Return the current user
GET	`/auth/google/start`	Start Google OAuth
GET	`/auth/google/callback`	Process the OAuth callback
POST	`/auth/forgot-password`	Request a password reset
POST	`/auth/reset-password`	Consume a reset token

Documents

Method	Path	Purpose
POST	`/upload`	Upload one document
POST	`/upload/batch`	Upload up to ten documents
GET	`/documents`	List the user's documents
GET	`/documents/{id}`	Get a document record
DELETE	`/documents/{id}`	Delete a document, optionally purging related conversation data

Query and sessions

Method	Path	Purpose
POST	`/query`	Non-streaming question-answering
POST	`/query/stream`	SSE question-answering
POST	`/query/image`	Image-driven question extraction and Q&A
POST	`/suggest`	Generate suggested questions
GET	`/sessions`	List sessions
GET	`/sessions/{id}`	Retrieve a session and messages
PATCH	`/sessions/{id}`	Rename a session
DELETE	`/sessions/{id}`	Delete a session
GET	`/sessions/{id}/export.md`	Export Markdown
GET	`/sessions/{id}/export.pdf`	Export PDF

Voice, feedback, and administration

Method	Path	Purpose
POST	`/transcribe`	Transcribe audio
POST	`/feedback/{message_id}`	Record message feedback
GET	`/admin/stats/summary`	Summary metrics
GET	`/admin/stats/timeseries`	Daily activity
GET	`/admin/stats/rag`	RAG evaluation aggregates
GET	`/admin/stats/recent`	Recent query records

SSE event contracts

The standard query endpoint uses JSON-enveloped events:

```
data: {"type":"step","data":{"state":"understand","status":"complete"}}
data: {"type":"token","data":{"token":"...","delta":"..."}}
data: {"type":"final","data":{"answer":"...","citations":[]}}
data: {"type":"done"}
```

The image endpoint uses named events:

```
event: questions
data: {"questions":[],"image_url":"..."}

event: answer_start
data: {"id":1}

event: answer_done
data: {"id":1,"answer":"...","citations":[]}

event: done
data: {}
```

The two streaming formats are documented as different and should be treated as separate contracts until unified.

13. Frontend structure

13.1 Routes

The documented routes include:

- `/` — public landing page.
- `/login` — login and registration.
- `/forgot-password` and `/reset-password` — password reset.
- `/auth/google/callback` — OAuth callback.
- `/app` — authenticated workspace.
- `/admin` — authenticated admin dashboard with an additional admin-role check.

13.2 Main components

The main workspace contains branding/header elements, a session list, document list, agent settings, index statistics, chat header, chat panel, message bubbles, answer cards, agent trace, suggested questions, and image attachment controls.

13.3 Streaming behavior

The SSE client reads the response stream, splits event frames, parses `data:` lines as JSON, and dispatches stage, token, final, completion, and error events to the application.

13.4 Mobile behavior

The documented mobile shell activates at widths up to 768px and includes:

- An off-canvas sidebar drawer.
- A sticky mobile header and backdrop.
- Body scroll locking while the drawer is open.
- Safe-area spacing for devices with display insets.
- Scroll anchoring to the most recent user message.
- A single-column message layout to prevent narrow text columns.
- A 16px minimum input font size to prevent iOS Safari auto-zoom.

14. Voice and image workflows

14.1 Voice input

The primary path uses the browser Web Speech API. A fallback records audio and submits it to the backend transcription endpoint using Groq Whisper. Cleanup should stop media tracks and release recognition/recording resources when the component unmounts.

14.2 Voice output

Browser SpeechSynthesis provides text-to-speech. Markdown is normalized before speech so formatting symbols, table syntax, and link markup are not read literally. The selected voice can be stored in local browser settings.

14.3 Image-assisted Q&A

The documented image endpoint uses a vision-capable model to extract questions and search queries from an uploaded image. Each extracted question is then passed through the document retrieval and answer-generation pipeline. The frontend presents a numbered set of answers and source citations.

The vision endpoint's event contract differs from the standard query stream. Image contents and extracted text must be treated as untrusted input and tested against prompt-injection attempts.

15. Admin dashboard and metrics

The admin dashboard is protected by frontend route checks and backend authorization. The documented sections include:

- **Summary:** counts of users, sessions, messages, and documents.
- **Time series:** signups, logins, and sessions over time.
- **RAG health:** grounding/faithfulness, context precision, verification pass rate, abstention rate, composite score, and latency.
- **Recent queries:** recent assistant turns with question previews, metrics, and abstention status.

Admin access is associated with an `is_admin` field. The source notes describe an optional `ADMIN_EMAIL` promotion behavior on first registration; verify this against the current code before relying on it.

16. Deployment

16.1 Documented hosting layout

The supplied configuration describes two Railway services:

- Frontend: static React bundle built with the configured frontend builder.
- Backend: FastAPI application launched with Uvicorn.
- Persistent volume: `/data`, used for SQLite and ChromaDB.

16.2 Environment configuration

Variable	Purpose
`ENV`	Environment mode
`FRONTEND_URL`	Public frontend origin
`GROQ_API_KEY`	Groq API credential
`JWT_SECRET`	JWT signing secret
`GOOGLE_CLIENT_ID`	OAuth client ID
`GOOGLE_CLIENT_SECRET`	OAuth client secret
`GOOGLE_REDIRECT_URI`	OAuth callback
`RESEND_API_KEY`	Email delivery credential
`RESEND_FROM_EMAIL`	Sender email
`RESEND_FROM_NAME`	Sender display name
`SQLITE_PATH`	SQLite database file
`CHROMA_PATH`	ChromaDB persistence directory
`VITE_API_BASE`	Frontend API base URL, resolved at build time

Never publish real environment values. Store secrets in the hosting provider's secret manager or environment configuration.

16.3 Health checks

The notes document `GET /health` for the backend and `/` for the frontend. Health checks should be confirmed in the deployed application after every release.

16.4 Production checklist

- ☐ Configure all required secrets.
- ☐ Confirm the frontend API base URL and backend origin.
- ☐ Validate CORS, OAuth redirect, and cookie settings.
- ☐ Confirm persistent volume mounts and writable paths.
- ☐ Test user registration, login, logout, and password reset.
- ☐ Test document upload, listing, retrieval, and deletion.
- ☐ Verify citations, unknown-ID handling, and abstention behavior.
- ☐ Test streaming disconnects and provider rate-limit errors.
- ☐ Confirm admin endpoints reject non-admin users.
- ☐ Review known security risks and document accepted residual risks.

17. Local development

Prerequisites

- Python 3.11.
- Node.js 18+ and npm.
- Groq API key.
- Optional Google OAuth credentials and Resend API key.

Backend

```
git clone https://github.com/Anas-Mirza-22/Argus-Grounded-Document-QA.git
cd Argus-Grounded-Document-QA/backend
python3.11 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

Create a `.env` file containing at least:

```
ENV=development
FRONTEND_URL=http://localhost:5173
GROQ_API_KEY=replace_with_your_key
JWT_SECRET=replace_with_a_long_random_secret
SQLITE_PATH=./argus.db
CHROMA_PATH=./chroma_data
```

Run the backend:

```
uvicorn app.main:app --reload --port 8000
```

Frontend

In another terminal:

```
cd Argus-Grounded-Document-QA/frontend
npm install
npm run dev
```

Open `http://localhost:5173`. For phone testing on the same Wi-Fi network, run `npm run dev -- --host` and open the network URL printed by Vite.

18. Engineering challenges and lessons

The supplied project notes describe the following implementation challenges and resolutions.

Challenge	Resolution / lesson
Citation chips appeared concatenated	Give citation rendering one owner and normalize marker spacing before rendering
Grounded paraphrases failed token-overlap checks	Compare normalized content tokens rather than all words
“CV” answers were rejected because “CV” was absent from text saying “resume”	Add carefully scoped exceptions for common file-type/acronym tokens
Mobile messages collapsed to a narrow strip	Update the grid template when hiding the avatar column
User question scrolled out of view during streaming	Anchor scrolling to the latest user message instead of the bottom
Sticky header covered the question	Reserve top spacing inside the scroll container
Body scroll remained locked after closing drawer	Restore the canonical empty overflow value on close/cleanup
Suggested questions exposed other users' document names	Resolve documents using the authenticated user's scoped records
OAuth cookie was lost in some browser flows	Introduce a short-lived exchange token and set the cookie from a fetch response
Current session summary appeared as past-session context	Exclude the active session from summary retrieval
Model weights exceeded GitHub file limits	Keep large binaries out of Git and add them to `.gitignore`

Challenge	Resolution / lesson
Frontend API calls returned HTML	Configure a deterministic backend URL rather than silently falling back to the frontend origin
Duplicate Documents heading	Assign ownership of the section heading to one component
Provider 429 errors looked like model defects	Route cheaper structured tasks to a smaller model and throttle requests
Desktop sidebar blocked mobile chat input	Build a mobile-specific drawer shell instead of shrinking the desktop layout

19. Known limitations and open risks

The following reflect the supplied project notes, not a live audit.

Product / operational limitations

- Incognito Chrome authentication may still fail in some configurations.
- Resend sandbox restrictions limit password-reset delivery until the sender domain is verified.
- Groq rate limits can temporarily interrupt requests.
- The frontend and image-query endpoints use different SSE formats.
- The deployment is designed for one backend replica.
- The current trace UI may not display all classification/adversarial fields.

Security risks identified in the notes

- Missing Origin/CSRF checks on `/auth/exchange`.
- Exchange tokens are not bound to the initiating client.
- Voice and vision endpoints need additional prompt-injection testing.
- Every endpoint must maintain strict per-user ownership checks.

Quality and operational gaps

- No golden evaluation set and prompt regression harness.
- No prompt version tracking.
- No CI pipeline listed in the supplied notes.
- Server-side query length and agent-setting caps need confirmation.
- Per-user quota tracking and adversarial-attempt analytics are planned.

20. Roadmap

High priority

1. Build a 20–30 example golden evaluation set and run it in pytest.
2. Add per-stage timing and a metrics endpoint.
3. Add CSRF/Origin validation and exchange-token client binding.
4. Unify SSE event contracts.
5. Add continuous integration for linting, type checking, and tests.
6. Enforce server-side limits for query length and agent settings.
7. Record prompt versions in query metadata.

Medium term

- Evaluate a single-origin deployment.
- Use Redis for shared rate limits and exchange-token storage if scaling.
- Consider PostgreSQL for multi-instance deployment.
- Expand prompt regression coverage.

Long term

- Stream citations as they are parsed.
- Add a multi-document comparison interface.
- Display additional trace information.
- Add admin analytics for adversarial attempts.

21. Glossary

Term	Definition
Abstention	Refusal to answer when evidence is insufficient
Agent trace	Visible record of pipeline stages for a request
Broad retrieval	Per-document search mode used for summaries and comparisons
Chunk	Text segment produced from a document
Citation coverage	Measure of how retrieved evidence is represented by citations
Content token	Meaning-bearing token retained after normalization and stopword removal
Exchange token	Short-lived, one-time token used to establish a session cookie
Filter-at-DB	Applying document scope inside the vector-store query
Grounding	Support for an answer in the retrieved source context
RAG	Retrieval-Augmented Generation
Session	Conversation thread with its own messages and title
Support check	Heuristic check for answer tokens absent from the retrieved text
Targeted retrieval	Flat search optimized for a specific question
Verification verdict	Combined result of grounding, citation, and model checks

22. Contribution guidelines

Changes should preserve the system's evidence-first behavior:

1. Do not silently answer from general knowledge when document evidence is missing.
2. Validate citations against retrieved source material.
3. Keep user and document access scoped by authenticated ownership.
4. Treat document contents, image text, and user prompts as untrusted input.
5. Fail closed when required verification cannot be completed.
6. Add regression tests for retrieval, citation parsing, abstention, security, and error paths.

Before opening a pull request, review the diff, run the tests available in the repository, and avoid committing secrets, private documents, database files, or model-weight binaries.

23. License

The supplied project notes do not identify a license. Add a `LICENSE` file and specify the selected license before distributing the project.